

Machinestein Files

Arip Asadulaev

February 9, 2026

1 $\mathcal{D}$ ual Advantage Fields

1.1 Research Proposal

Describe your idea in a few sentences.

We propose an algorithm for efficient policy extraction from dual goal representation. Starting from the dual goal representations in [PML25], where the goal-conditioned potential is parameterized by an inner product $V_\theta(s, g) \equiv f(\psi_\theta(s), \phi_\theta(g)) = \psi_\theta(s)^\top \phi_\theta(g)$, we show that the goal embedding $\phi_\theta(g)$ is exactly the gradient of the learned potential V with respect to the state representation coordinates: $\nabla_\psi V_\theta(s, g) = \phi_\theta(g)$. Using this, we derive a *dual advantage* estimator that enough for policy iteration.

1.2 Motivation

Describe the problem you want to solve. Is it important?

Dual goal representations encode a goal through its temporal relations to all states and provide appealing theoretical properties (e.g., sufficiency and noise invariance). However, the practical algorithm trains a structured inner-product temporal-distance/value proxy for representation learning **and then commonly relies on a separate downstream offline GCRL loop for policy extraction**. This two-stage design reflects a conceptual mismatch: a temporal-distance or value field is a *global scalar map*, whereas policy improvement fundamentally depends on *local action comparisons*. Moreover, [PML25] noted that directly **extracting a policy from the inner product V failed** because the value surface was too rigid. Therefore, connecting dual representations to advantage computation is important for (i) more direct policy improvement, (ii) potentially simpler pipelines (less reliance on a second value learner), and (iii) better utilization of the structure already imposed by dual spaces.

1.3 Related Work

What is available on this topic? Papers first, then implementations and everything else. For each paper, a short summary to clarify its focus, results, reliability, and reproducibility.

Dual goal representations. [PML25] define the dual goal representation $\phi^\vee(g)$ as the functional mapping each state to the optimal temporal distance to g , and show theoretical sufficiency and noise invariance. Practically, they approximate the temporal-distance function via a parameterization $d^*(s, g) = f(\psi(s), \phi(g))$ with an inner product $f(\psi, \phi) = \psi^\top \phi$, and use goal-conditioned IQL to learn this proxy. **Their Algorithm 1 then trains a downstream goal-conditioned policy $\pi(a \mid s, \phi(g))$ with an arbitrary offline GCRL method. They further report that directly extracting a policy from the structured inner-product function can degrade performance, motivating the separate downstream loop.**

Policy improvement using advantages. [DS95] argue that policy iteration should only require relative measures of action utility (advantages), not absolute values. In their continuous deterministic setting, they define an advantage $A^w(x, u) = r(x, u) + f(x, u) \cdot \nabla_x V^w(x)$ and improve policies by maximizing A^w . They emphasize that learning the relevant derivatives can be underconstrained and introduce conservative-field (curl-free) structure as an additional constraint. This peer-reviewed NeurIPS paper is conceptually reliable; translating its differential constraints to modern deep, discrete-time offline RL requires careful design.

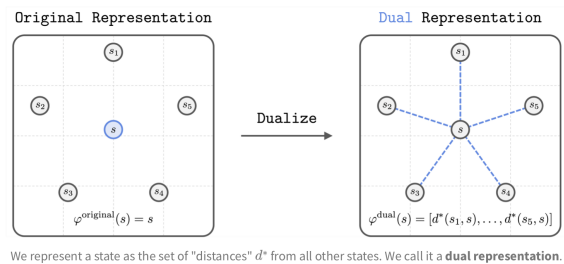


Figure 1: Illustration from the original paper on dual representations

1.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

Step 1: Fix the representation class We adopt [PML25]’s practical instantiation:

$$V_\theta(s, g) \triangleq f(\psi_\theta(s), \phi_\theta(g)) = \psi_\theta(s)^\top \phi_\theta(g), \quad (1)$$

where $\psi_\theta : \mathcal{S} \rightarrow \mathbb{R}^N$ and $\phi_\theta : \mathcal{S} \rightarrow \mathbb{R}^N$ are state and goal heads, and V_θ is trained as a goal-conditioned temporal-distance/value proxy using an offline value-learning algorithm (goal-conditioned IQL).

Step 2: Prove (in the model class) that $\nabla_\psi V_\theta(s, g) = \phi_\theta(g)$. The key identity follows from basic multivariate calculus and the linearity of the chosen aggregator. Fix any goal g . Define $V_\theta(\cdot, g)$ as a function of the representation vector $\psi \in \mathbb{R}^N$:

$$\tilde{V}_\theta(\psi; g) \triangleq \psi^\top \phi_\theta(g) = \sum_{i=1}^N \psi_i \phi_{\theta,i}(g). \quad (2)$$

For each coordinate $j \in \{1, \dots, N\}$, the partial derivative is

$$\frac{\partial \tilde{V}_\theta(\psi; g)}{\partial \psi_j} = \frac{\partial}{\partial \psi_j} \sum_{i=1}^N \psi_i \phi_{\theta,i}(g) = \phi_{\theta,j}(g), \quad (3)$$

because $\phi_\theta(g)$ does not depend on ψ , and $\partial \psi_i / \partial \psi_j = \mathbb{I}[i = j]$. Collecting these partial derivatives yields the gradient vector:

$$\nabla_\psi \tilde{V}_\theta(\psi; g) = \left[\frac{\partial \tilde{V}_\theta}{\partial \psi_1}, \dots, \frac{\partial \tilde{V}_\theta}{\partial \psi_N} \right]^\top = [\phi_{\theta,1}(g), \dots, \phi_{\theta,N}(g)]^\top = \phi_\theta(g). \quad (4)$$

Since $V_\theta(s, g) = \tilde{V}_\theta(\psi_\theta(s); g)$, the identity $\nabla_\psi V_\theta(s, g) = \phi_\theta(g)$ holds everywhere in representation space by construction of the inner-product model.

Intuition: This identity provides the critical geometric bridge between the two papers. It reveals that [PML25]’s goal representation $\varphi(g)$ is not merely a latent embedding; it is precisely the *gradient* of the value function in latent space. By learning the dual representation, we are implicitly learning the derivative ∇V required by [DS95].

Step 3: Derive a dual advantage formula. [DS95] express advantages as reward plus an inner product between a dynamics direction and a value gradient. In discrete-time goal-conditioned RL, define the standard advantage:

$$A(s, a, g) \triangleq Q(s, a, g) - V(s, g). \quad (5)$$

Using [PML25]’s TD-style target for the learned Q head in Algorithm 1, $Q(s, a, g) \approx r(s, g) + \gamma V(s', g)$, and substituting $V(s, g) = \psi(s)^\top \phi(g)$, we obtain

$$\begin{aligned} A(s, a, g) &\approx r(s, g) + \gamma \psi(s')^\top \phi(g) - \psi(s)^\top \phi(g) \\ &= r(s, g) + (\gamma \psi(s') - \psi(s))^\top \phi(g). \end{aligned} \quad (6)$$

By (4), $\phi(g)$ is the gradient of the potential in ψ -coordinates, and $\gamma \psi(s') - \psi(s)$ is the action-conditional direction of change in those coordinates. Equation (6) is therefore the discrete, representation-space analogue of [DS95]’s advantage formula.

Intuition: This formula represents the exact discrete analogue of [DS95]’s advantage equation $A = r + \dot{x} \cdot \nabla V$. Here, $\gamma \psi(s') - \psi(s)$ represents the “change in state” vector, and $\varphi(g)$ represents the gradient. We have successfully substituted the abstract gradient term from [DS95] with the learned goal representation from [PML25], unifying the two approaches.

Step 4: Practical estimator and policy improvement. We can say [PML25] provide the *map* (a potential), while the dual advantage provides the *compass* (directional action scores). So, to avoid relying on a sampled s' at decision time, we learn an action-effect head $u_\xi(s, a) \approx \mathbb{E}[\gamma\psi(s') - \psi(s) \mid s, a]$ by regression on offline transitions. We then define

$$\hat{A}(s, a, g) \triangleq r(s, g) + u_\xi(s, a)^\top \phi_\theta(g), \quad (7)$$

and perform policy improvement by selecting actions that maximize $\hat{A}$, consistent with the advantage-driven improvement principle in [DS95]. Optionally, we regularize u_ξ with Dayan-inspired cycle/conservative-field constraints to improve generalization beyond actions heavily represented in the dataset.

Justification: [PML25] noted that directly extracting a policy from the inner product V failed because the value surface was too rigid. Our approach solves this by using $\varphi(g)$ as a local *compass* (gradient) rather than a global map. Instead of asking “which state has the highest inner product?” (rigid), we ask “which action moves me in the direction of $\varphi(g)$?” (flexible). This aligns the action selection with the local advantage landscape, rather than the global value approximation.

1.5 Discussion

Why would it work? What conclusion and new intuition we gets. *Bonus: why it may not work?*
What we should pay attention to?

The identity $\nabla_\psi V_\theta(s, g) = \phi_\theta(g)$ shows that Park’s goal embedding is not merely a descriptor of a target state; it is the *dual* object that evaluates local directions in representation space. Consequently, combining $\phi(g)$ with an action-effect vector yields an advantage signal that is inherently local and action-comparative, precisely aligned with advantages suffice for policy iteration.

Potential failure modes include: (i) representation collapse or misalignment, where ψ does not parameterize controllable directions well; (ii) stochastic or multi-modal transitions, where a mean action-effect u_ξ may be insufficient; and (iii) offline extrapolation, where maximizing $\hat{A}$ exploits approximation error. [DS95]’s analysis suggests that additional structural constraints (e.g., conservative-field/cycle conditions) can be important when the directional information is underdetermined, motivating explicit regularization of the learned action-effect field.

1.6 Experiments A

How to quickly test an idea to see if it works (conventionally, in a couple weeks) to decide if it’s worth exploring further.

Implement a minimal extension of [PML25]’s Algorithm 1: learn (ψ, ϕ) with the same inner-product value proxy and then train $u_\xi(s, a)$ by regressing to $\gamma\psi(s') - \psi(s)$ on the offline dataset. Compare: (i) Park dual representations + downstream GCRL (baseline) versus (ii) Dual-Advantage using $\hat{A}(s, a, g) = r(s, g) + u_\xi(s, a)^\top \phi(g)$ and an advantage-weighted actor update. Evaluate on a small subset of OGBench tasks, reporting success rates and variance across seeds. Key diagnostic: correlation between $\hat{A}$ and empirical short-horizon returns for held-out (s, g) pairs. Ablate the cycle/conservative regularizer inspired by [DS95].

1.7 Experiments B

If Experiment A is successful, what are the next steps?

Scale evaluation to the full benchmark suite used by [PML25] and test whether dual advantages can reduce dependence on a separate downstream value learner without degrading performance. Extend to pixel-based settings, where [PML25] note late-fusion limitations for representation-conditioned policies, and assess whether a state-aware variant of the dual advantage (e.g., learning u_ξ from joint state-goal features) addresses these issues.

2 Compositional Inverse Reinforcement Learning

2.1 Research Proposal

Describe your idea in a few sentences.

We propose *inverse subtask discovery*: learn a **decomposed reward** from expert demonstrations, where each learned component corresponds to a minimal, interpretable subtask. Concretely, we parameterize the reward as an additive mixture of nonnegative basis functions and infer them via maximum-entropy inverse reinforcement learning (IRL). We then *solve* more efficiently and robustly by using the learned components for shaping, curriculum, and modular policy learning.

2.2 Motivation

Describe the problem you want to solve. Why is it important?

Many tasks have a single global reward (often sparse/terminal) while the optimal behavior has clear internal structure: prerequisite objectives, enabling transitions, collecting resources, satisfying constraints, and so on. With only a monolithic reward, learning can be brittle and data-hungry, and the learned behavior is hard to interpret. We want an algorithm that can *infer the subtasks from data* rather than requiring hand-designed hierarchies. If we can reliably discover a small set of reward components that explain expert behavior, we gain interpretable structure and reusable building blocks for generalization, transfer, and faster learning.

2.3 Related Work

What is available on this topic? List papers first, then implementations and other resources. For each paper, include a short summary clarifying its focus, results, reliability, and reproducibility.

Maximum-entropy inverse reinforcement learning (MaxEnt IRL). Models demonstrations via a soft-optimal policy where trajectory probability is proportional to the exponential of discounted return, yielding an occupancy-matching gradient. Provides a principled likelihood objective and differentiable learning signal for reward inference [ZMB⁺08, WOP15].

Hierarchical RL and option discovery. HRL methods assume or learn temporally-extended skills/subgoals; they often require design choices about hierarchy or sequencing. In contrast, inverse subtask discovery focuses on decomposing reward explanations without imposing a fixed hierarchy or temporal ordering.

2.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

We observe expert demonstrations and estimate the expert discounted state occupancy $\hat{d}(s)$. Instead of learning a single reward, we assume the task reward can be expressed as an additive mixture of K latent subtasks:

$$R(s) = \sum_{k=1}^K w_k r_k(s),$$

where $r_k(s) \geq 0$ are learned reward components and $w_k \geq 0$ are their weights.

MaxEnt IRL objective. Under maximum-entropy IRL, trajectories are distributed as $p(\tau | R) \propto \exp\left(\sum_{t \geq 0} \gamma^t R(s_t)\right)$, inducing a soft-optimal policy π_R and occupancy $d^{\pi_R}(s)$. We fit the decomposition

by optimizing a demonstration likelihood objective plus structural regularization. Using the occupancy-gradient property of MaxEnt IRL, learning reduces to driving the model occupancy toward the expert occupancy through the learned basis functions:

$$\min_{\{r_k\}, \{w_k\}} \sum_{k=1}^K w_k \left(\mathbb{E}_{d^{\pi_R}}[r_k(s)] - \mathbb{E}_{\hat{d}}[r_k(s)] \right) + \mathcal{R}(\{r_k\}, \{w_k\}).$$

Structural regularization for interpretability. Reward decompositions are highly non-identifiable without constraints. We therefore enforce: (i) nonnegativity of w_k and $r_k(s)$ (additive subtasks, no cancellation), (ii) normalization of each basis (e.g., unit ℓ_2 norm) to remove scale ambiguity, (iii) sparsity penalties to localize each subtask in state space, and (iv) overlap penalties so multiple bases do not explain the same states.

Understand, then solve. After learning $\{r_k\}$ and $\{w_k\}$, we use the components as a task “parse”: - as *shaping rewards* to accelerate learning from sparse signals, - as *subgoal detectors* for curriculum and exploration, - as *modular objectives* to train reusable skills/policies, - and as *diagnostic explanations* of which subtasks drive behavior.

2.5 Discussion

Why would it work? What conclusions, intuitions, and insights do we get? *Bonus: why might it not work, and what should we pay attention to?*

Why it should work. MaxEnt IRL provides a clean learning signal: if the reward explains expert behavior, the induced occupancy should match the expert occupancy. Decomposing the reward pushes the model to explain occupancy patterns using multiple components. With sparsity and overlap penalties, components are encouraged to become localized and distinct, corresponding to minimal subtasks rather than arbitrary algebraic factors. Empirically, this yields components that align with intuitive prerequisites (e.g., key $\rightarrow$ switch $\rightarrow$ door $\rightarrow$ goal) even when the environment reward is purely terminal.

Why it may not work / what to watch. Non-identifiability does not disappear; the regularizers only bias the solution. Components can still collapse, fragment, or encode nuisance structure if the demonstrations are limited or ambiguous. Interpretability depends on the choice of regularizers and K : too small yields entangled subtasks; too large yields redundant fragments.

2.6 Experiments A

How can you quickly test the idea (e.g., within a couple of weeks) to decide if it is worth exploring further?

Two-week sanity check (replication). Implement the decomposed MaxEnt IRL objective in a small deterministic gridworld with conditional dynamics and a terminal reward only. Generate expert demonstrations from a soft-optimal policy. Train $K = 4$ reward bases with (i) nonnegativity, (ii) basis normalization, and (iii) sparsity + overlap penalties. Evaluate: (1) whether each learned r_k localizes to semantically meaningful regions (e.g., key/switch/door/goal), (2) redundancy/overlap scores between components, (3) policy reconstruction: does π_R reproduce expert occupancy? Ablate the regularizers to confirm they prevent degenerate decompositions.

2.7 Experiments B

If Experiment A is successful, what are the next steps?

To be discussed.

3 Quasi-Metric Generative Models

3.1 Research Proposal

Describe your idea in a few sentences.

A novel framework that integrates Quasimetric principles into generative flow models. The core idea is to learn a global, asymmetric distance function (a quasimetric) that quantifies the “generation effort” or “cost-to-go” between any noisy state and the clean data manifold. By optimizing this metric to satisfy the triangle inequality while maximizing the separation between noise and data, we can derive optimal, shortest-path generative trajectories and enable adaptive-step solvers that allocate computational budget based on the semantic complexity of the generation step, rather than fixed time intervals.

3.2 Motivation

Describe the problem you want to solve. Is it important?

Current state-of-the-art generative models [HJA20, LCBH⁺22, DLL⁺26], suffer from a “geometric blindness.” They typically rely on Euclidean metrics (L2 norm) or fixed-step Ordinary Differential Equation (ODE) solvers (e.g., Euler or Heun steps) to traverse the path from noise to data [WD25]. This approach assumes that the difficulty of generation is uniform across the entire trajectory, which is physically and semantically incorrect.

Generating a coherent structure from noise is fundamentally **asymmetric**: it is computationally expensive to create order from chaos (generation), while it is trivial to destroy order (adding noise). Standard symmetric metrics cannot capture this directionality. Imposing symmetric metrics on asymmetric dynamics leads to failure to capture complex reachability structures.

Furthermore, the lack of an intrinsic “progress metric” leads to inefficient sampling. A solver might waste steps on simple background rendering while rushing through complex feature formation. Solving this is crucial for reducing the computational cost of inference (NFE) and improving the quality of generated samples by adhering to the true geometry of the data manifold.

3.3 Related Work

What is available on this topic? Papers first, then implementations and everything else. For each paper, a short summary to clarify its focus, results, reliability, and reproducibility.

1. Optimal Goal-Reaching Reinforcement Learning via Quasimetric Learning (QRL):

This is the foundational paper for our proposal. Wang et al. demonstrate that the optimal value function $V^*(s, g)$ in goal-reaching problems is geometrically equivalent to a quasimetric space [WTIZ23].

- **Focus:** The paper introduces an objective function that maximizes the distance between states $d_\theta(s, g)$ subject to local transition constraints.
- **Key Result:** They prove that this objective recovers the true shortest path (optimal value) without the need for bootstrapping or temporal-difference learning, which stabilizes training.
- **Relevance:** This provides the mathematical justification for treating “cost-to-go” (or generation steps) as a geometric distance

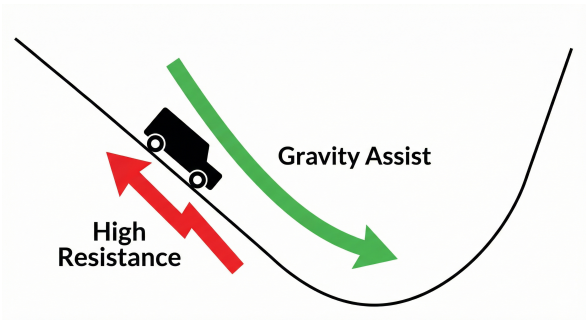


Figure 2: Illustration of quasimetric role in physical (RL) world

that satisfies the triangle inequality but is not symmetric.

2. Flow Matching for Generative Modeling:

Flow Matching (FM) is a simulation-free approach to training Continuous Normalizing Flows. It regresses a vector field $v_t(x)$ to map a source distribution (noise) to a target distribution (data) [LCBH⁺22].

- **Focus:** Training continuous-time generative models via simple regression objectives.
- **Limitation:** Standard FM does not learn a metric of progress; it only learns the local velocity field. It lacks a global sense of “how far” the sample is from the target.

3.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

We propose to treat the generative process as a goal-reaching problem where the “goal” is the data manifold and the “state” is the noisy latent representation. We will train a **Quasimetric Critic** [WTIZ23] alongside the standard Flow Matching vector field.

Mathematical Formulation

Let $v_\phi(x, t)$ be the Flow Matching vector field (the actor) and $d_\theta(x, y)$ be the quasimetric value function (the critic). We adapt the QRL objective to the continuous domain of diffusion:

1. The Global Objective (Stretching):

We want to maximize the estimated distance between the source distribution (noise, x_1) and the target distribution (data, x_0). This corresponds to the “maximal spreading” principle in QRL.

$$\mathcal{L}_{global} = -\mathbb{E}_{x_1 \sim p_{noise}, x_0 \sim p_{data}} [d_\theta(x_1, x_0)] \quad (8)$$

Intuitively, we pull the noise and data as far apart as possible in the quasimetric embedding. The “tautness” of this distance will represent the minimum effort required to generate the data.

2. The Local Constraint (Consistency):

The distance function must be consistent with the actual flow dynamics. In QRL, the distance between state s and next state s' is bounded by the transition cost $-r$. In continuous flow, the distance between x_t and x_{t-dt} must be bounded by the magnitude of the vector field:

$$\mathcal{L}_{constraint} = \mathbb{E}_{t,x} \left[\text{ReLU} (d_\theta(x_t, x_{t-dt}) - \|v_\phi(x_t)\| \cdot dt)^2 \right] \quad (9)$$

This ensures that the learned distance d_θ never overestimates the physical capability of the flow model to transport mass.

Adaptive Quasimetric Sampling

Once d_θ is trained, we utilize it to perform **Quasimetric Adaptive Sampling**. Instead of a fixed step size dt , the ODE solver selects the next step size dynamically. The solver attempts to take the largest step Δt such that the change in quasimetric distance remains within a semantic error tolerance ϵ :

$$|d_\theta(x_t, x_{target}) - d_\theta(x_{t+\Delta t}, x_{target})| \approx \epsilon$$

This allows the model to traverse “easy” regions (where d_θ changes slowly or predictably) rapidly and slow down in “difficult” regions (where the manifold curvature or complexity is high), effectively optimizing the computational budget.

3.5 Discussion

Why would it work? What conclusion and new intuition we gets. *Bonus: why it may not work?*
What we should pay attention to?

Why it works: The approach rests on the **Value-Quasimetric Equivalence Theorem** proven in the QRL paper. It states that the optimal cost-to-go function is exactly a quasimetric. By explicitly modeling this geometry, we enforce the triangle inequality, which acts as a powerful inductive bias. This prevents the model from learning inconsistent shortcuts or circular paths in the latent space. The “stretched chain” analogy suggests that this method will naturally find the straightest possible paths (geodesics) through the highly curved manifold of natural images.

The learned quasimetric $d_\theta(x, y)$ provides a measure of semantic distance. If $d_\theta(x_{dog}, x_{cat})$ is small, the model implies a low-cost transformation exists between them. If it is large, they are semantically distinct. Critically, because it is a *quasimetric*, we preserve the arrow of time: $d(noise, image)$ is finite, but $d(image, noise)$ can be learned to be different, reflecting the thermodynamic asymmetry of the process. **Risks and Attention Points:**

- **Dimensionality:** The QRL paper experiments on low-dimensional states (MountainCar, Maze2D). Generative models operate in pixel space. Learning a global quasimetric in high dimensions is notoriously difficult and may suffer from sparse gradients.
- **Constraint Satisfaction:** The strict ReLU constraint is hard to enforce perfectly with neural networks. If the constraint is violated, the distance metric collapses (the chain breaks).
- **Optimization Stability:** Maximizing the distance d_θ (the global objective) can lead to diverging weights if not carefully balanced with the local constraint, as noted in QRL’s implementation details regarding the Lagrange multiplier.

3.6 Experiments A

How to quickly test an idea to see if it works (conventionally, in a couple weeks) to decide if it’s worth exploring further.

Toy 2D Flow Matching:

- **Dataset:** Use the “2D Swirl” or “8-Gaussians” toy datasets commonly used in literature.
- **Baselines:** Standard Flow Matching with fixed Euler steps ($N=20$).
- **Method:** Train a small MLP to learn the quasimetric d_θ using the QRL objective alongside the flow vector field.
- **Evaluation:**
 - **Path Linearity:** Plot the generated trajectories. Are the QFM trajectories straighter (lower curvature) than standard FM?
 - **Metric Consistency:** Verify the triangle inequality. Pick three random points x, y, z and check if $d(x, z) \leq d(x, y) + d(y, z)$.
 - **Asymmetry Check:** Verify if the model learns the arrow of time by comparing $d(noise, data)$ vs $d(data, noise)$.

If the QFM trajectories are straighter or the adaptive solver can produce equivalent quality 2D densities with 30% fewer steps, the idea is validated.

3.7 Experiments B

If Experiment A is successful, what are the next steps?

Standard generative modeling datasets.

4 $\mathcal{M}$ ass Moving Games

4.1 Research Proposal

Describe your idea in a few sentences.

We propose *Mass Moving Games (MMGs)*: a unified view in which learning, inference, and control are all framed as **moving mass on a directed graph under flow-conservation constraints** while optimizing an objective over flows. The key observation is that the constraints in (10) are essentially *GFlowNet flow-matching constraints* (with sources/sinks playing the role of reward injection/absorption), and closely related to *occupancy-measure RL*. MMGs treat the graph (explicit or implicit), the conservation law, and the objective as modular components, enabling BOT-like branching, GFlowNet-style amortized sampling.

4.2 Motivation

Describe the problem you want to solve. Why is it important?

Gilbert’s BOT objective can be written as

$$\arg \min_{B, E, x_B, m \in E} \sum_{(i,j) \in E} m_{ij}^\alpha \|x_i - x_j\|_2, \quad (10)$$

$$\begin{aligned} \text{supply} \quad \mu_s &= \sum_k m_{sk} - \sum_k m_{ks} && \text{at each source } s, \\ \text{demand} \quad \mu_t &= \sum_k m_{kt} - \sum_k m_{tk} && \text{at each sink } t, \\ \text{conservation} \quad \sum_k m_{kb} &= \sum_k m_{bk} && \text{at each branching point (BP) } b. \end{aligned}$$

(Here $m = \{m_{ij}\}_{(i,j) \in E}$ are nonnegative edge flows; B are branching nodes with coordinates x_B [LFSH22].)

BOT is a beautiful but difficult object: it optimizes not only the flows m_{ij} but also the network structure (edges E) and even geometry (branching point locations x_B). The conservation constraints are clean and local, but the global search over networks is complicated.

GFlowNets [BJK⁺21], meanwhile, solve a seemingly different problem –sampling complex objects with probability proportional to a reward –yet they rely on nearly the *same conservation law*: “incoming flow equals outgoing flow” at intermediate nodes. The practical miracle is that GFlowNets do *not* need to learn or enumerate the full underlying state graph; they learn just the *flows that matter* through local policies and sampled trajectories. MMGs is use that property for BOT-like problems (and beyond): keep the conservation structure, plug in different objectives (transport cost, reward, energy), and avoid explicitly constructing the “true” graph.

4.3 Related Work

What is available on this topic? List papers first, then implementations and other resources. For each paper, include a short summary clarifying its focus, results, reliability, and reproducibility.

Branched Optimal Transport (BOT). Uses diminishing marginal costs (e.g., m^α with $\alpha \in (0, 1)$) so merging flows is beneficial, yielding branched networks. Classic formulations optimize over graph structure and sometimes continuous branch-point geometry, subject to supply/demand and conservation constraints (as in (10)).

GFlowNets (flow matching / trajectory balance). Learn forward (and optionally backward) policies on a huge implicit DAG/graph of partial constructions. Their defining constraint is conservation

of flow at each intermediate state, with special boundary conditions at sources and terminals (terminal inflow matches reward). Training is local and sample-based, so the full graph never needs to be enumerated.

Occupancy-measure RL / linear programming views of RL. RL can be written as optimizing a reward objective over a visitation distribution subject to Bellman/flow constraints. Conceptually, this is “mass moves under dynamics + conservation,” just with discounting and stochastic transitions [ZMB⁺08].

4.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

1) One conservation law for everything. The three constraints in (10) are just node-wise conservation with boundary injections/absorptions. Define a net injection b_v per node:

$$b_v = \begin{cases} +\mu_s & v \in S, \\ -\mu_t & v \in T, \\ 0 & v \in B. \end{cases}$$

Then conservation can be written compactly as

$$\sum_{j:(v,j) \in E} m_{vj} - \sum_{i:(i,v) \in E} m_{iv} = b_v \quad \forall v \in V.$$

This is the structural bridge: *the constraint is the same object* across BOT, GFlowNets, and occupancy-measure RL; what changes is the interpretation of b_v and the objective.

2) MMG objective: In Mass Moving Games $m \geq 0$ satisfying node-wise conservation (and any extra constraints). and a chosen objective $\mathcal{J}(m)$ (or reward $\mathcal{R}(m)$) is:

$$\min_{m \in \mathcal{F}} \mathcal{J}(m), \quad \mathcal{F} = \{m \geq 0 : Am = b\},$$

where A is the node-edge incidence operator. BOT is recovered by $\mathcal{J}(m) = \sum_{(i,j)} \|x_i - x_j\|_2 m_{ij}^\alpha$. The MMG move is to allow different $\mathcal{J}$:

- **RL:** set m to be an occupancy/flow and maximize $\sum m \cdot r$ (possibly with entropy regularization).
- **GFlowNets:** define terminal rewards $R(x)$ and enforce terminal inflow equals $R(x)$ (so samples are proportional to R).
- **BOT/network design:** use concave edge costs to encourage branching/merging.

3) Amortized flow learning without learning the “true graph.” In BOT, the “true” optimal transport network is unknown and can be combinatorially complex. In GFlowNets, the underlying state graph is typically astronomically large (all partial objects), yet training avoids explicit enumeration. MMGs import this trick: We represent the graph *implicitly* via a **construction process** (states are partial networks/partial routings; actions add an edge, add a branching point, connect components, etc.). We learn a **local policy** (forward transitions) that determines how mass flows through this implicit graph. We train it using **local conservation constraints** (flow matching / trajectory balance), which only requires samples from visited states.

Result: we never need to “learn the true graph” as an explicit object. We learn the *necessary flows* concentrated on useful regions of the implicit graph: high-reward (or low-cost) completions and the trajectories that reach them. Concretely, a standard parameterization is

$$m_{s \rightarrow s'} = F(s) P_F(s' | s),$$

where $F(s)$ is the total flow through state s and P_F is a stochastic forward policy. Conservation is enforced locally (inflow + injection = outflow), and terminal inflow is tied to a reward or cost-based weight (e.g., $R(x) = \exp(-\beta \mathcal{J}(x))$).

4.5 Discussion

Why would it work? What conclusions, intuitions, and insights do we get? *Bonus: why might it not work, and what should we pay attention to?*

Why it should work. The conservation constraints are the easy part: they are local, compositional, and enforceable by learning rules that do not require global graph knowledge. That is exactly why GFlowNets scale: they replace “solve a global constrained problem on a huge graph” with “learn local transitions so that flow is conserved on the states you actually visit.” MMGs use the same principle: *learn the right flows, not the whole graph*. The MFG viewpoint adds a second layer of intuition: BOT-like branching is what you get when many infinitesimal agents face flow-dependent costs with economies of scale. The “branching” is not a special network primitive; it is an equilibrium pattern of a mean-field-coupled flow system.

Why it may not work / what to watch. Concave costs ($\alpha < 1$) can make optimization landscapes tricky; amortized policies may prematurely collapse to brittle solutions without careful regularization. Also defining a construction process that is expressive enough to represent good networks, yet local enough that learning is stable.

4.6 Experiments A

How can you quickly test the idea (e.g., within a couple of weeks) to decide if it is worth exploring further?

Two-week sanity check: implicit-graph BOT via amortized flows. Create a small routing task on a grid/planar graph with multiple sources and sinks. Compare:

- **Classical flow optimization** on the explicit graph (baseline).
- **MMG-GFlowNet:** treat a full routing/network as an object; use reward $R = \exp(-\beta\mathcal{J})$; train with flow-matching/trajectory-balance.
- **MMG-RL:** train a policy to minimize cost under conservation constraints (occupancy-measure style), with/without entropy regularization.

Sweep α from 1 (linear OT-like) to < 1 (branching regime). Metrics: objective value, constraint violation, branching statistics (edge support / junction count), and sample diversity (for the GFlowNet variant).

4.7 Experiments B

If Experiment A is successful, what are the next steps?

To be discussed.

5 $\mathcal{E}$ cho-State Reasoning Hypotheses

5.1 Research Proposal

Describe your idea in a few sentences.

We propose a unifying framework for understanding reasoning. Idea is to connect three mechanisms: latent reasoning, reservoir computing (nonlinear random or frozen state dynamics), and autostigmergy (externalizing memory into the environment or intermediate traces). We argue that reasoning effectiveness should be measured not only by task performance, but by where task-relevant information is stored and how robust that storage is to targeted perturbations.

5.2 Motivation

Describe the problem you want to solve. Is it important?

It is unknown how and why the reasoning, especially latent reasoning, works. Meanwhile the reasoning algorithms became the main fuel for the progress in AI.

5.3 Related Work

What is available on this topic? Papers first, then implementations and everything else. For each paper, a short summary to clarify its focus, results, reliability, and reproducibility.

Latent iterative reasoning models (HRM / TRM). Recent work on Hierarchical Reasoning Models and the Tiny Recursive Model [JM25] formalizes reasoning as iterative refinement with persistent latent states. These models distinguish a latent solution representation from a latent reasoning state (z_L) that functions as an internal scratchpad and is carried across steps to prevent forgetting of intermediate reasoning. Empirically, deep recursion with small networks yields strong generalization, suggesting that learned dynamics and parameter efficiency are central.

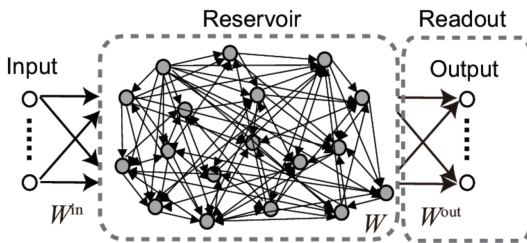


Figure 3: Reservoir computer architecture

Reservoir computing and echo-state networks. Classic reservoir computing demonstrates that random recurrent dynamics, when stable, can provide rich nonlinear features and temporal memory, enabling strong performance with only a trained readout [YHB⁺24, BGGB21]. This line of work explains why frozen recurrent cores can still support nontrivial computation, but also highlights limits on structured, algorithmic reasoning without task-shaped dynamics.

Autostigmergy in reinforcement learning. [DFF⁺19] study interval timing in deep RL agents and show that feedforward agents without internal

memory can solve temporal tasks by using stereotyped action trajectories, effectively storing time in the external environment (“the environment as a clock”). They term this strategy *autostigmergy*, contrasting it with recurrent agents that develop internal timer-like representations. They further show that frozen LSTMs can still perform well, and that truncating credit assignment disproportionately harms feedforward agents, underscoring distinct memory mechanisms. The task and code are open-sourced, supporting reproducibility.

5.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

Reasoning fundamentally requires memory, and memory can be implemented in multiple substrates. Latent reasoning works when internal state is persistent and trainable; reservoir computing explains why even random dynamics can help by providing memory and nonlinear features; autostigmergy explains why, in the absence or costliness of internal memory, agents externalize state into the world.

We model reasoning systems as iterative dynamical systems that require persistent state. This state are: (i) trained internal latents (latent reasoning), (ii) untrained but persistent internal dynamics (reservoirs), or (iii) external traces created through interaction with the environment or text (autostigmergy) [DFF⁺19] (Fig 6). Our key proposal is to treat these substrates as interchangeable resources that optimization can trade off, rather than as qualitatively distinct phenomena. Introduce a set of metrics that explicitly quantify reliance on each substrate: **(i) External erasure tests:** remove or randomize intermediate external traces (text, tool outputs, environment state) between reasoning steps while preserving internal latents. **(ii) Internal lesion tests:** reset, noise, or bottleneck internal latent states (e.g., z_L) while leaving external traces intact. **(iii) Reservoir ablations:** freeze recurrent cores at random initialization and train only readouts or solution updates, isolating the contribution of generic nonlinear dynamics. From these interventions, we define quantitative indices of internalization versus externalization, enabling principled comparison across architectures and tasks.

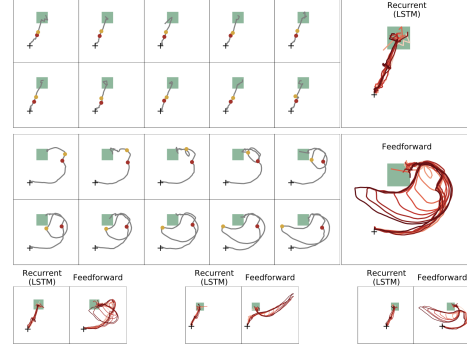


Figure 4: Autostigmergy with FeedForward networks (reach me for details).

5.5 Discussion

Why would it work? What conclusion and new intuition we gets. *Bonus: why it may not work? What we should pay attention to?*

Potential failure is the entanglement of memory with compute depth in recurrent systems. Very careful control experiments are required.

5.6 Experiments A

How to quickly test an idea to see if it works (conventionally, in a couple weeks) to decide if it's worth exploring further.

We propose a small-scale experimental matrix on an iterative reasoning benchmark. (i) Baseline model with trained recurrence and intact latent reasoning. (ii) External-trace erasure between steps. (iii) Internal-latent reset or noise injection every k steps. (iiii) Frozen recurrent core with only readout/solution update trained. We measure performance drops under each condition and compute normalized internalization and externalization scores. Success is indicated by systematic, interpretable tradeoffs across conditions rather than uniform degradation.

5.7 Experiments B

If Experiment A is successful, what are the next steps?

To be discussed.

6 Convex Energy-based Reasoning

6.1 Research Proposal

Describe your idea in a few sentences.

We propose *Convex Compositional Energy Minimization*: learn **input-convex** energy functions for small reasoning subproblems and compose them at test time by summation. By parameterizing each subproblem energy with an Input Convex Neural Network (ICNN)[[AXK17](#)], the composed energy remains convex in the decision variables, yielding a globally well-behaved landscape and inference via convex optimization rather than heuristic search.

6.2 Motivation

Describe the problem you want to solve. Is it important?

Compositional energy-based reasoning generalizes by combining energies of many constraints at inference time, but the resulting objective can become highly nonconvex, creating local minima and requiring heavy sampling machinery [[OD25](#)]. We aim to make compositional reasoning *optimization-friendly* by enforcing convexity in the solution variables, so that adding more constraints increases difficulty mainly through problem size, not through pathological energy geometry. This matters for reliable generalization to larger instances (more clauses, bigger boards/graphs) under fixed training data.

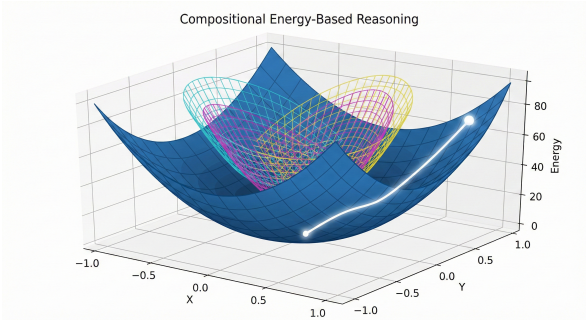


Figure 5: Composition of convex functions to build a global convex energy landscape.

6.3 Related Work

What is available on this topic? Papers first, then implementations and everything else. For each paper, a short summary to clarify its focus, results, reliability, and reproducibility.

Input Convex Neural Networks [[AXK17](#)]. Introduces scalar-valued neural energies constrained to be convex in a subset of inputs, enabling inference by solving $\arg \min_y f(x, y; \theta)$ as a convex program. Provides nonnegative hidden-to-hidden weights, convex nondecreasing activations.

Generalizable Reasoning through Compositional Energy Minimization [[OD25](#), [DLM20](#)]. Learns energies over subproblems and composes them at test time by summing multiple energy functions, enabling additional constraints and larger instances. Because composition yields complex nonconvex landscapes, the paper introduces Parallel Energy Minimization (PEM) to mitigate local minima and improve sampling quality. The code is released in a public repository for both.

6.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

Let a reasoning instance be decomposed into subproblems $x = \{x_1, \dots, x_N\}$ sharing a global decision variable y (or blocks y_k with coupling). We learn subproblem energies $E_\theta^k(x_k, y)$ such that valid local satisfaction yields low energy. We enforce that each E_θ^k is convex in y by using a (P)ICNN architecture, where x_k can enter through a non-convex “context” path while convexity is preserved in y .

At test time we solve

$$\hat{y} \in \arg \min_{y \in \mathcal{Y}} \sum_{k=1}^N E_{\theta}^k(x_k, y) + R(y),$$

where $\mathcal{Y}$ is a convex relaxation (e.g., $[0, 1]^n$ or a product of simplices) and $R(y)$ encourages discrete structure (e.g., entropy barriers or simplex constraints). Since sums of convex functions are convex, the composed objective is convex in y , enabling global optimization (projected gradient, bundle methods, or off-the-shelf convex solvers). We train by (i) local supervision on subproblems and (ii) optional end-to-end fine-tuning via differentiating through the convex argmin (implicit differentiation / KKT-based gradients), aligning global minima with full-instance solutions.

6.5 Discussion

Why would it work? What conclusion, intuition and insights we gets. *Bonus: why it may not work? What we should pay attention to?*

Why it should work. Compositional energy minimization generalizes by adding constraints through energy summation; ICNNs guarantee that adding energies does not introduce additional nonconvex traps in the decision space. Thus, harder instances can be attacked by increasing optimization accuracy or tighter relaxations, rather than relying on stochastic exploration to escape local minima. The resulting intuition is that *generalization-by-composition becomes a convex program*, shifting difficulty from “energy geometry”.

Why it may not work / what to watch. Convexity is restrictive: a convex relaxation may yield fractional solutions that decode poorly, and convex energies may underfit highly multimodal discrete landscapes. The main risks are degenerate energies (flat objectives), and mismatch between convex minima and discrete optima.

6.6 Experiments A

How to quickly test an idea to see if it works (conventionally, in a couple weeks) to decide if it’s worth exploring further.

Two-week sanity check: replicate a small compositional benchmark (e.g., 3-SAT with clause-wise decomposition, or N-Queens with row/col/diag factors) using convex relaxations. Train a per-constraint ICNN energy E_{θ}^k on subproblem data only, compose $\sum_k E_{\theta}^k$ at test time, and run convex inference. Compare against (a) a nonconvex MLP energy with the same composition and (b) the PEM-based baseline from compositional EBM work. Study the tradeoff between convexity strength and expressivity by moving from fully convex ICNN to PICNN and to partially convex hybrids

6.7 Experiments B

If Experiment A is successful, what are the next steps?

If Experiment A is positive: (i) extend to graph coloring with edge-wise energies on simplex-relaxed node colors. Benchmark compute scaling: convex solver iterations vs particle count (PEM) at fixed wall-clock.

7 Zero-shot Order Book Dynamics Model (Bonus)

7.1 Research Proposal

Describe your idea in a few sentences.

We propose a *zero-shot execution foundation model* that decouples **market dynamics** from **trader preferences**. Using Successor Measures / Forward-Backward (FB) representations, we learn a universal “forward” representation of the limit order book dynamics once, and adapt instantly to new objectives (risk aversion, urgency) by changing a low-dimensional preference vector rather than retraining a policy.

7.2 Motivation

Describe the problem you want to solve. Is it important?

Execution problems in algorithmic trading vary continuously with client preferences: a risk-averse client wants inventory tightly controlled, while an urgent client prioritizes fast completion. Classical stochastic-control models allow “turning a knob” (e.g., a risk aversion parameter γ), but rely on restrictive assumptions such as Brownian price dynamics. Deep RL can learn from realistic, high-dimensional limit order book (LOB) data, but is typically brittle to reward changes: changing the objective often requires full retraining.

The goal is to get the best of both worlds: (i) *zero-shot adaptation* like classical models, but (ii) with a learned dynamics model capable of capturing non-Gaussian microstructure.

7.3 Related Work

What is available on this topic? Papers first, then implementations and everything else. For each paper, a short summary to clarify its focus, results, reliability, and reproducibility.

Successor measures / Forward-Backward representations [TO21]. Introduces a factorization of value functions that separates dynamics from task preferences. Instead of learning a single scalar value tied to one reward, one learns a representation of *future state occupancy* (a successor object), and adapts to new tasks via linear scalarization.

Avellaneda–Stoikov market making [AS08]. A classical stochastic control benchmark assuming Brownian mid-price dynamics and CARA utility. It yields closed-form structure such as the reservation price and spread adjustments as functions of risk aversion γ , providing a clean “zero-shot” control law under strong assumptions.

Universal value function ideas / reward-conditioned RL. A broad line of work conditions value/policy on a goal or reward parameter. The FB viewpoint is a particularly structured instance: it explicitly learns an object tied to dynamics (successor measure/features) and makes preference adaptation a linear operation.

Implementations. Classical AS implementations are lightweight but rely on stylized models. Deep RL implementations for execution/LOB modeling exist but are typically trained per objective. This work proposes architectures that train a single FB model and then adapt via preference vectors at inference.

7.4 Idea

How do you propose to solve the problem? What is new about this solution? Give a fairly detailed description.

1) Decouple dynamics and preferences via Successor Measures. For a policy π , define the (discounted) *successor measure* as the occupancy measure over future states:

$$M^\pi(s, a, ds') = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \delta(s_t \in ds') \mid s_0 = s, a_0 = a \right].$$

Intuitively, M^π captures *where you will end up* under the market dynamics and policy, independent of what you care about.

2) Forward-Backward (FB) factorization for zero-shot objectives. Consider a family of tasks parameterized by a preference vector $z \in \mathbb{R}^d$ with linear reward

$$r_z(s) = z^\top \psi(s),$$

for some state features $\psi(s)$. The FB framework posits that the optimal action-value can be approximated bilinearly as

$$Q^*(s, a; z) \approx F(s, a)^\top z,$$

where $F(s, a)$ is a learned *forward representation* encoding task-agnostic reachability (successor-feature-like quantities), and z encodes task desirability (client preference). This is the mechanism that enables **zero-shot adaptation**: change z at inference time, keep F fixed.

3) The isomorphism: Avellaneda–Stoikov is a special case of FB. The Avellaneda–Stoikov objective can be rewritten as a scalarization of a vector objective by defining a feature vector

$$\phi(s, q) = \begin{bmatrix} s \\ -q^2 \end{bmatrix},$$

and a task vector determined by risk aversion γ :

$$z = \begin{bmatrix} 1 \\ \frac{1}{2}\gamma\sigma^2(T-t) \end{bmatrix}.$$

Then the AS value has the structure $V_{AS}(s, q) \propto \phi(s, q)^\top z$, i.e., a linear preference over a fixed feature representation. In this view, AS is a “zero-shot” controller because the dynamics-dependent term is solved analytically (under Brownian assumptions) and preference enters through $z(\gamma)$.

4) Method A: Fully learned deep FB (LOB foundation model). To handle heavy tails and microstructure, learn $F_\theta(s, a)$ from LOB observations using a deep encoder (e.g., Transformer). Learn an auxiliary state encoder $B_\xi(\cdot)$ to generate a “virtual reward” from next-state representations. Training samples random preference vectors z and minimizes a bootstrapped Bellman error:

$$\mathcal{L}(\theta, \xi) = \mathbb{E}_{\tau, z} \left[F_\theta(s_t, a_t)^\top z - (r_t(z) + \gamma F_{\theta^-}(s_{t+1}, \pi_z(s_{t+1}))^\top z) \right]^2,$$

with $r_t(z) = z^\top B_\xi(s_{t+1})$. The important design choice is that training does *not* commit to one reward; it trains across a *distribution over preferences*.

5) Method B: “Brownian-FB” hybrid (physics-informed prior). When price efficiency approximately holds, enforce an analytic forward component for mid-price p and learn only the microstructure:

$$F(s, a) = F_{\text{Analytic}}(p) \oplus F_{\text{Learned}}(m, a),$$

where $s = (p, m)$ splits mid-price and microstructure features. The analytic layer is derived from the diffusion resolvent / Green’s function and implemented as a fixed kernel over anchor prices:

$$F_{\text{Analytic}}(p)_i = \exp(-\alpha|p - \text{anchor}_i|).$$

At inference, the user specifies γ and the agent computes a preference-weighted action value

$$Q(s, a) = w_{\text{price}}^\top F_{\text{Analytic}}(p) + w_{\text{micro}}^\top F_{\text{Learned}}(m, a),$$

where w_{micro} is derived from γ to adjust inventory/risk sensitivity without retraining.

7.5 Discussion

Why would it work? What conclusion, intuition and insights we gets. *Bonus: why it may not work? What we should pay attention to?*

Why it should work. The key insight is that “what the market will do next” (dynamics) and “what the trader wants” (utility/preferences) are separable objects. Successor measures/features are designed to encode dynamics in a reusable way; linear scalarization via z then provides a clean and fast knob for preference changes. The Avellaneda–Stoikov mapping provides a sanity check: under classical assumptions, the same structural factorization yields a known zero-shot policy. The proposed deep FB model is the nonlinear, data-driven generalization: learn the forward dynamics representation from LOB, then adapt to new γ -like objectives by changing z .

Why it may not work / what to watch. Zero-shot hinges on the task family being representable by the chosen feature/preference parameterization (e.g., rewards linear in ψ). If real objectives require richer nonlinear utilities, the factorization may underfit. Training must also cover a sufficiently rich distribution of preference vectors z ; otherwise “zero-shot” becomes extrapolation. Finally, markets are nonstationary: a universal dynamics representation can drift, so one must monitor stability and consider continual learning or domain adaptation.

7.6 Experiments A

How to quickly test an idea to see if it works (conventionally, in a couple weeks) to decide if it’s worth exploring further.

Two-week sanity check: implement Method A in a small execution simulator with synthetic LOB dynamics (or a replay-based simulator from historical data). Train a single FB model with randomized preference vectors z (spanning risk/urgency). At evaluation, freeze the model and sweep z (or γ -mapped z) *without retraining*. Measure: (1) realized cost/slippage vs. urgency, (2) inventory variance vs. risk preference, (3) stability of the policy under objective changes compared to a standard RL baseline that requires retraining for each z .

7.7 Experiments B

If Experiment A is successful, what are the next steps?

If Experiment A is positive: (i) evaluate on real LOB data across assets/regimes (crypto vs equities) and test distribution shift, (ii) benchmark Method B (Brownian-FB) for regimes where diffusion priors are reasonable, comparing sample efficiency and safety, (iii) test richer preference parameterizations (beyond a single γ) such as multi-objective tradeoffs (execution cost, inventory risk, fill probability), (iv) investigate calibration and risk constraints (e.g., CVaR-style penalties) while preserving fast preference adaptation.

References

- [AS08] Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008.
- [AXK17] Brandon Amos, Lei Xu, and J Zico Kolter. Input convex neural networks. In *International conference on machine learning*, pages 146–155. PMLR, 2017.
- [BJK⁺21] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation. *Advances in Neural Information Processing Systems*, 34:27381–27394, 2021.
- [DFF⁺19] Ben Deverett, Ryan Faulkner, Meire Fortunato, Gregory Wayne, and Joel Z Leibo. Interval timing in deep reinforcement learning agents. *Advances in Neural Information Processing Systems*, 32, 2019.
- [DLL⁺26] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026.
- [DLM20] Yilun Du, Shuang Li, and Igor Mordatch. Compositional visual generation with energy based models. *Advances in Neural Information Processing Systems*, 33:6637–6647, 2020.
- [DS95] Peter Dayan and Satinder Singh. Improving policies without measuring merits. *Advances in neural information processing systems*, 8, 1995.
- [GBGB21] Daniel J Gauthier, Erik Bollt, Aaron Griffith, and Wendson AS Barbosa. Next generation reservoir computing. *Nature communications*, 12(1):5564, 2021.
- [HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [JM25] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.
- [LCBH⁺22] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [LFSH22] Peter Lippmann, Enrique Fita Sanmartín, and Fred A Hamprecht. Theory and approximate solvers for branched optimal transport with multiple sources. *Advances in Neural Information Processing Systems*, 35:267–279, 2022.
- [OD25] Alexandru Oarga and Yilun Du. Generalizable reasoning through compositional energy minimization. *arXiv preprint arXiv:2510.20607*, 2025.
- [PML25] Seohong Park, Deepinder Mann, and Sergey Levine. Dual goal representations. *arXiv preprint arXiv:2510.06714*, 2025.
- [TO21] Ahmed Touati and Yann Ollivier. Learning one representation to optimize all rewards. *Advances in Neural Information Processing Systems*, 34:13–23, 2021.
- [WD25] Runqian Wang and Yilun Du. Equilibrium matching: Generative modeling with implicit energy-based models. *arXiv preprint arXiv:2510.02300*, 2025.
- [WOP15] Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum entropy deep inverse reinforcement learning. *arXiv preprint arXiv:1507.04888*, 2015.
- [WTIZ23] Tongzhou Wang, Antonio Torralba, Phillip Isola, and Amy Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. In *International Conference on Machine Learning*, pages 36411–36430. PMLR, 2023.
- [YHB⁺24] Min Yan, Can Huang, Peter Bienstman, Peter Tino, Wei Lin, and Jie Sun. Emerging opportunities and challenges for the future of reservoir computing. *Nature Communications*, 15(1):2056, 2024.

- [ZMB⁺08] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.